

Guide d'étude : visualisation avec R

Ce qu'il faut savoir faire pour les oraux et l'évaluation sur table

Ce que ce module ajoute

Le module 6 vous a appris à importer, ranger et transformer des données. Ce module les **montre**. Un graphique sert à deux choses : **explorer** (voir les valeurs aberrantes, les **NA**, les groupes mal codés, les distributions bizarres que les moyennes cachent) et **communiquer** (convaincre un lecteur, dans un article ou une présentation).

ggplot2 trace ce qu'on lui demande, sans vérifier que c'est ce qu'on voulait. Les erreurs de ce module sont donc presque toutes **silencieuses** : une couleur fixe écrite dans **aes()**, des barres qui additionnent au lieu de faire la moyenne, des données retirées par **limits** avant le calcul d'un boxplot, une ligne qui zigzague faute de **group**, des catégories dans l'ordre alphabétique au lieu de l'ordre du trimestre, une légende de R de base qui ne correspond pas aux couleurs. Au mieux, un *warning* le signale. Un assistant d'IA écrit le code d'un graphique en une seconde, mais il ne **regarde** pas le graphique produit. C'est sur ces erreurs que portent les questions.

Comment ce module est évalué

Comme le reste du cours, **en présence et sans outil d'IA** :

- à l'**évaluation sur table** (50 % de la note du cours) : [À PRÉCISER : sur papier ou en salle informatique surveillée, durée, documents permis] ;
- lors des **oraux** individuels (25 % chacun) : [À PRÉCISER : quel oral porte sur la visualisation].

On ne vous demandera pas de mémoriser les noms de toutes les fonctions ni leurs arguments. On vérifiera que vous savez **lire** un code **ggplot2** et décrire le graphique qu'il produit, **prédire** ce que donne une modification, **repérer** pourquoi un graphique est faux ou trompeur, et **choisir** le graphique adapté à une question. Ce sont les formats des pauses « À vous » des diapos, des sections « Et si... ? » des TP et de l'exercice formatif.

Les types de questions

Type	Ce qu'on vous demande	Exemple déjà vu
Prédire	Décrire le graphique (ou l'erreur) sans exécuter	<code>ggplot(iris, aes(Sepal.Length)) + geom_point()</code> (diapos)
Erreur	Expliquer pourquoi un graphique est faux	<code>geom_col()</code> qui additionne les deux lignes « Citron » (diapos, TP1)
Et si	Anticiper l'effet d'une modification	Ajouter <code>scale_y_continuous(limits = c(50, 100))</code> (TP2)
Expliquer	Justifier un concept avec vos mots	<i>Mapping</i> ou <i>setting</i> (diapos)
Lire	Interpréter un graphique	Un volcano plot : effet, significativité, seuils (diapos, TP3)
Écrire	Écrire le code qui produit un graphique donné	Pause « À vous : quel code a produit ce graphique ? » (diapos)
Choisir	Proposer le graphique adapté	Barplot de la moyenne ou points ? (diapos, TP3)

À l'oral, chaque réponse est suivie d'une **relance** : « pourquoi ? », « comment l'auriez-vous vérifié ? ». Une réponse juste sans explication ne suffit pas.

Ce que vous devez savoir faire

V-BAS : les graphiques de R de base

- Prédire le graphique que produit `plot()` selon le type de ses arguments (facteur, numérique, deux numériques, facteur + numérique).
- Utiliser `hist()`, `boxplot(y ~ groupe)`, `barplot(table(x))`, et ajouter des couches (`lines()`, `points()`, `legend()`).
- Sauvegarder avec `png()` ... `dev.off()`.
- Expliquer pourquoi une légende écrite à la main peut être **fausse sans message**, et l'écrire avec `levels()`.

Où le travailler : section 2 des diapos (pause « À vous : quel graphique ? »), TP1 (Q1 à Q3, « Et si... ? » 3).

V-GRA : la grammaire de `ggplot2`

- Nommer les trois ingrédients d'un graphique : **données** (un tableau), **mapping** (`aes()`), **geom**.
- Lire un code `ggplot2` et décrire le graphique ; écrire le code d'un graphique décrit ou dessiné.
- Savoir que le `+` termine la ligne, et que les couches se dessinent dans l'ordre.

- Choisir un geom selon le nombre et le type des variables.

Où le travailler : section 3 des diapos (pause « À vous : prédire »), TP1 (Q4, Q5), exercice formatif Q1.

V-MAP : mapping ou setting

- Distinguer une propriété qui **dépend d'une colonne** (dans `aes()`, avec légende) d'une **valeur fixe** (hors de `aes()`).
- Prédire ce que donnent `aes(color = "blue")` (une donnée, pas une couleur) et `geom_point(color = colonne)` (une erreur).
- Savoir qu'un `aes()` dans `ggplot()` vaut pour toutes les couches, et dans un geom pour cette couche seulement (une ou trois droites de régression).

Où le travailler : section 3 des diapos (« Piège : une constante dans `aes()` », « Où placer `aes()` ? »), TP1 (Q5, « Et si... ? » 1 et 2), exercice formatif Q2.

V-STA : statistiques et positions

- Distinguer `geom_bar()` (compte les **lignes**) et `geom_col()` (trace **y** tel quel).
- Savoir que `geom_col()` **empile** les lignes de même **x** : la barre est une **somme**, jamais une moyenne ; résumer d'abord avec `summarise()`.
- Lire les positions `"stack"`, `"dodge"`, `"fill"` (l'axe y devient une proportion).
- Superposer données et résumé avec `stat_summary()`.

Où le travailler : section 4 des diapos (séquences `geom_bar()` et « Citron »), TP1 (Q6, « Et si... ? » 4 et 5), TP2 (Q7), exercice formatif Q3.

V-ECH : échelles et coordonnées

- Lire le nom d'une échelle (`scale_ + aesthetic + type`) ; choisir des couleurs avec un **vecteur nommé**.
- Savoir que `ggplot2` ne lit pas le contenu : la catégorie « Rouge » ne devient pas rouge.
- Expliquer pourquoi `limits`, `xlim()` et `ylim()` **retirent des données** avant le calcul (boxplots faux, `warning Removed n rows`), alors que `coord_cartesian()` zoome.
- Savoir qu'un axe logarithmique retire les zéros (`log10(0) = -Inf`), et utiliser `log2(x + 1)`.

Où le travailler : section 4 des diapos (« Piège : `limits` retire des données », pause « À vous »), TP2 (« Et si... ? » 3), TP3 (« Et si... ? » 3), exercice formatif Q4.

V-HAB : facettes, thèmes, titres, sauvegarde

- Utiliser `facet_wrap()` et `facet_grid()` ; savoir que `scales = "free_y"` rend les panneaux difficiles à comparer.
- Donner un titre à chaque axe (avec ses **unités**) et à chaque légende avec `labs()` ; placer ou retirer la légende.
- Sauvegarder avec `ggsave()` en fixant la taille ; savoir que sans `plot =`, c'est le **dernier** graphique qui est sauvegardé.

Où le travailler : section 4 des diapos, TP1 (Q4, Q7), TP2 (Q2, Q5).

V-DON : préparer les données

- Reconnaître qu'un tableau large doit passer en **format long** (`pivot_longer()`) pour qu'une variable aille sur un axe ; prédire le nombre de lignes obtenu.
- Ordonner les catégories : ordre alphabétique par défaut ; `factor(levels = )`, `fct_inorder()`, `fct_reorder()`, `fct_infreq()`.
- Utiliser `group` (ou `color`) pour tracer une ligne par groupe ; reconnaître une ligne qui **zigzag**.
- Transformer une année ou un code numérique en catégorie avec `factor()`.
- Lire les *warnings* sur les **NA** et repérer une catégorie **NA** dans une légende.

Où le travailler : section 5 des diapos, TP2 (Q4 à Q6, « Et si... ? » 1, 2, 4 et 5), exercice formatif Q5 et Q6.

V-BIO : les graphiques de la bioinformatique

- Lire un boxplot (médiane, quartiles, moustaches) et savoir qu'il cache l'effectif.
- Construire et lire un **volcano plot** : $\log_2$ fold change en x, $-\log_{10}(p_{adj})$ en y, seuils choisis par l'analyste ; trouver dans quelle condition un gène est plus exprimé en vérifiant sur les données.
- Lire un MA plot, un histogramme de p-values, un Q-Q plot de p-values.
- Faire une **heatmap** avec `pheatmap()` : pourquoi `log2(x + 1)`, pourquoi `scale = "row"`.

Où le travailler : section 6 des diapos, TP3, exercice formatif Q7.

V-CHX : choisir et critiquer un graphique

- Choisir le graphique selon la question (comparer, distribution, relation, évolution) et le type de variables.
- Expliquer les limites du barplot de moyennes pour des données continues avec peu de réplicats (Weissgerber *et al.*, 2015) ; préférer les points.
- Repérer un axe tronqué, une 3D inutile, deux axes y, des couleurs rouge/vert, un histogramme dont la largeur de classe cache un groupe.

Où le travailler : section 6 des diapos (pauses « À vous : qu'est-ce qui ne va pas ? »), TP3 (Q8, « Et si... ? » 5).

V-AUD : vérifier un graphique

- Repérer dans un script `ggplot2` les erreurs **silencieuses** : `limits` qui retire des données, ordre alphabétique, somme présentée comme moyenne, constante dans `aes()`.
- Vérifier un graphique en **refaisant le calcul** avec `summarise()` et en lisant les axes, les légendes et les *warnings*.
- Distinguer ce qui est faux de ce qui est correct.

Où le travailler : activité « Auditer un script `ggplot2` généré par IA » et son corrigé.

Comment vous préparer

1. **Refaites sans ordinateur** les pauses « À vous », les « Et si... ? » des TP et l'exercice formatif : écrivez votre prédiction, *puis* vérifiez dans RStudio.
2. Devant chaque graphique, prenez l'habitude de répondre à trois questions : **que représente chaque axe** (et dans quelle unité) ? **combien d'observations** derrière chaque barre, boîte ou point ? **que cache** ce graphique ?
3. Devant chaque code, cherchez ce qui est **dans** `aes()` et ce qui est **hors** de `aes()`, et quelle **statistique** calcule chaque geom.
4. Lisez **tous** les *warnings*: `Removed n rows` signifie que des données ne sont pas sur le graphique.
5. Refaites l'activité d'audit **seul**, en appliquant les vérifications du corrigé. Les solutionnaires des TP donnent les réponses : lisez surtout les explications.

